

Architecture and trade-offs

A functional core with a thin shell. Replay is a pure fold — state and one instruction in, new state and one outcome out — with no clock, no randomness and no input or output anywhere inside it. The journal of instructions is the source of truth. The ledger book, the hold registry and the day reports are projections: discard any of them, replay the journal, and they rebuild identically. That is what makes append-only a structural property rather than a promise somebody has to keep.

The single most important rule in the design is that **no account object holds a balance**. A balance is derived from the entry list on demand, from a value day and a knowledge day. If an account had a total, something would eventually set it, and every guarantee below would become a convention instead.

1. Append-only at a hundred times the volume

What breaks first is the balance query. `LedgerBook.balanceAsOf` walks every entry in the book on every call, filtering by account and by two dates. The day close calls it several times per account per day, and every authorization decision calls it again. Cost grows with the product of history length and query count, so it degrades quadratically as a window lengthens rather than linearly with volume.

At the scale here — a handful of accounts, six days, tens of entries — that is free and the clarity is worth more. At a hundred times the volume it is the first thing to fall over, and it falls over on the read path, which is the path a customer is waiting on.

Where unbounded state accumulates. Three places, in order of severity. The entry list grows without limit and is never compacted, by design. The hold registry retains every authorization the replay has ever seen, including terminal ones, because a settled authorization that vanished would make the day it settled unreadable. Day reports retain every outcome for every day.

The cheapest structural change is a balance snapshot, keyed by account and value day, taken at each day close and written as another projection. A query then sums from the nearest snapshot at or before the requested value day rather than from the beginning of time, turning a full scan into a bounded one. It requires no change to the journal and no change to the append-only property, because a snapshot is derived data that can be thrown away and rebuilt — which is the test of whether a cache is safe to add here.

The complication that makes this more than a cache: a backdated entry invalidates every snapshot at or after its value day. Those have to be recomputed, and until they are, a query answered from a stale snapshot is wrong rather than slow. The honest version of this change therefore includes snapshot invalidation on backdated arrival, and that is the part that would need to be got right before anything else.

2. Value-dated entries in a licensed bank

Value dating is the feature in this design with the largest operational surface, and almost none of that surface is visible in the code.

A balance already reported to a customer can change. The scenario contains an instruction that arrives three days after the day it takes value from, and it moves a day that had already closed from positive to negative. Anyone who saw that day's balance in between was told something that is no longer true. In a licensed institution that has consequences well past the ledger: statements already issued,

interest and fees already assessed against the superseded figure, regulatory reporting already submitted for a period that has now moved, and — if the restated day happens to cross a reporting boundary — a submitted return that is retrospectively wrong.

Two dates means two versions of every question. "What was the balance on day two" has one answer for a customer dispute, another for a regulator asking what was reported at the time, and a third for a risk model asking what is true now. This design refuses to guess between them by making both dates mandatory at every call site. That refusal is cheap in code and is the main reason the query signature is deliberately awkward.

The control to add before go-live: a back-valuation gate. Any instruction whose value day falls before the last closed day should be quarantined rather than applied — held in a pending state, surfaced to an operator with the restatement it would cause and the downstream assessments it would change, and released only on explicit approval, with the approver and the reason recorded in the journal alongside the instruction.

Automatic application, which is what this system does today, is right for a replay harness and wrong for a bank. The gate is the difference between a correction and a silent rewrite of a day a customer has already been told about. It is also the natural place to attach the fee restatement question, which is the known failing test: whether reopening a day re-assesses its fees is an operational decision with an owner, not a branch in a policy class.

3. How an authorization ends

Six ways, of which the scenario exercises two. All six are modelled, because the set of terminal states is the part of an authorization lifecycle that gets discovered in production.

Ending	The real situation	What the system does
Settled	The merchant captures. The amount may differ from what was reserved	Books a debit for the settled amount, closes the authorization, releases any unused reservation
Partial settlement	A capture for less than was held — a split shipment, an unavailable item	Same as settled: the difference is released rather than kept reserved, because the transaction is complete
Reversed	The merchant cancels before capturing; the customer abandons the transaction	Releases the hold, closes as reversed, books nothing — no money ever moved
Expired	No capture arrives within the scheme's window	Modelled as a state and unreachable here: no expiry rule was given and inventing a period would be a number with no source
Released on account closure	The account is closed while a reservation is live	Every live hold is released and the closure records which ones, so a reservation cannot outlive the account it constrains
Never resolved inside the window	The window ends with the authorization still live	It stays live and is reported as live on the final day. It is not quietly cleaned up: an unresolved reservation is a fact about the account, not an untidiness

The distinctions are kept rather than collapsed into a single "closed" state. Settled and expired leave the same available balance behind and mean entirely different things to whoever has to explain the statement — and to whoever is measuring how many authorizations a merchant lets lapse.

The decision itself is a predicate over three numbers — ledger balance, active holds, proposed hold — with no access to a clock, a registry or the book. That isolation is what makes the scenario's most

important property testable directly: the same authorization, moved one position earlier in the stream, is approved rather than declined, because the balance in front of it is different. Order is load-bearing, and the test says so in one line.

A hold moves no money. It reduces available balance and leaves the ledger balance untouched, which is why holds live in their own registry rather than in the book. Nothing about a reservation belongs in a journal of movements.

4. What was cut, and the risk each cut defers

Every simplification below was a decision, not an omission. Naming them is stronger than half-building them.

Double-entry conservation. Entries are single-sided: a credit has no matching debit anywhere, because no contra account was given. Nothing structurally sums to zero, so the ledger cannot be proved consistent the way a real one is. *Deferred risk:* a bug that creates money is not caught by construction. *Mitigation:* the conservation check — each account's entries must sum to its final balance — runs in the suite and in the harness, and the harness exits non-zero on failure. That is narrower than double entry and it is what is available.

Persistence. Everything is in memory and dies with the process. *Deferred risk:* the entire durability question — write ordering, fsync, recovery, the journal being the thing that survives. The design is friendly to it, because the journal is already the source of truth and everything else is derived, but "friendly to" is not "solved".

Concurrency. The replay is single-threaded and nothing is synchronised. *Deferred risk:* the hardest problem in the real version. Two authorizations judged simultaneously against the same balance can both be approved when only one should be, and no amount of care inside the fold prevents that — it needs a serialisation point per account.

A calendar. Days are ordinals. *Deferred risk:* no timezone, no weekend rule, no holiday calendar, no cut-off time. Value dating in a real bank is defined against a banking calendar, and every one of those absent rules is a place where a real value date differs from this one.

Multi-currency accounts. An account has one currency for life. *Deferred risk:* an account holding two currencies is a different product with different rules, and none of them are here. The fee configuration already refuses to convert between currencies rather than pretending it can.

Hold expiry. The state exists; no rule produces it. *Deferred risk:* funds stay reserved indefinitely with nothing to release them, which is a customer-visible failure rather than an accounting one.

Idempotency on retried instructions. A duplicate event id is refused, which is idempotency by identifier and nothing more. *Deferred risk:* a genuine retry of a payment that arrives with a fresh id is indistinguishable from a second payment, which is exactly the case a real system has to get right.

Back-valuation and fee restatement. The known failing test. Fees are assessed forward-only, so a backdated instruction charges on the day it arrived rather than the day it overdrew. *Deferred risk:* the fee lands on a different day from the overdraft that caused it, which is visible to a customer reading a statement. Closing the gap needs the restatement run and the approval gate described above; `RestatementFeePolicy` in test scope shows the arithmetic and, notably, shows that the restating answer produces three fees rather than one — so the gap is not simply an unfinished feature with an obvious completion.

Snapshotting. Named in section 1 as the answer to the volume question rather than built. Premature over six days, and naming an omission deliberately is worth more than building something with nothing to say about it.

Patterns, and two deliberate absences

Event sourcing throughout. Strategy for the three policies, which is what lets a rejected alternative be implemented in test scope and measured rather than argued with. Builder for the configuration and the event stream. Immutable records for every value. A factory method pair on `Money`, split because one factory could not both refuse a typo and accept a legitimate rounding. A template method fixing the day-close stage order in a `final` method. A facade at `LedgerEngine`. Registries for accounts and holds. A refusal is a returned value rather than a thrown exception. A type-safe enum for rejection reasons.

A sealed interface with an exhaustive `switch` replaces the visitor: the same completeness guarantee — a new event kind fails to compile until every handler deals with it — without the double dispatch.

Not used, on purpose. *Singleton*, which would put shared mutable state behind a global and break the determinism the whole design rests on. *Observer*, which would introduce ordering ambiguity in the one place where ordering is the entire problem.